Bug 1

```
PROBLEMS   OUTPUT   TERMINAL   PORTS                                   bash - Week 11 Lab Assignment

amydayana@amy:~/eece2140/Week 11 Lab Assignment$ valgrind --leak-check=full --track-origins=yes ./part_c
==9354== Memcheck, a memory error detector
==9354== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==9354== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==9354== Command: ./part_c
==9354==
[3: Castiel] -> [2: Dean] -> [1: Sam] -> null
==9354==
==9354== HEAP SUMMARY:
==9354==     in use at exit: 144 bytes in 3 blocks
==9354==   total heap usage: 5 allocs, 2 frees, 74,896 bytes allocated
==9354==
==9354== 144 (48 direct, 96 indirect) bytes in 1 blocks are definitely lost in loss record 3 of 3
==9354==    at 0x4846FA3: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==9354==    by 0x10A65B: NodeList::addNode(int, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >) (part_c.cpp:76)
==9354==    by 0x10A4AD: main (part_c.cpp:120)
==9354==
==9354== LEAK SUMMARY:
==9354==    definitely lost: 48 bytes in 1 blocks
==9354==    indirectly lost: 96 bytes in 2 blocks
==9354==      possibly lost: 0 bytes in 0 blocks
==9354==    still reachable: 0 bytes in 0 blocks
==9354==         suppressed: 0 bytes in 0 blocks
==9354==
==9354== For lists of detected and suppressed errors, rerun with: -s
==9354== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
amydayana@amy:~/eece2140/Week 11 Lab Assignment$
```

Bug 2

```
File   Edit   Selection   View   Go   Run   Terminal   Help        Week 11 Lab Assignment [WSL: Ubuntu]

PROBLEMS   OUTPUT   TERMINAL   PORTS                                   bash - Week 11 Lab Assignme

amydayana@amy:~/eece2140/Week 11 Lab Assignment$ g++ -g -o part_c part_c.cpp
amydayana@amy:~/eece2140/Week 11 Lab Assignment$ valgrind --leak-check=full --track-origins=yes ./part_c
==11206== Memcheck, a memory error detector
==11206== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==11206== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==11206== Command: ./part_c
==11206==
==11206== Invalid read of size 8
==11206==    at 0x1095F9: NodeList::freeChain(ListNode*) (part_c.cpp:31)
==11206==    by 0x109681: NodeList::~NodeList() (part_c.cpp:44)
==11206==    by 0x1094A5: main (part_c.cpp:100)
==11206==  Address 0x4e1e188 is 40 bytes inside a block of size 48 free'd
==11206==    at 0x484A61D: operator delete(void*, unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==11206==    by 0x109494: main (part_c.cpp:97)
==11206==  Block was alloc'd at
==11206==    at 0x4846FA3: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==11206==    by 0x1096AB: NodeList::addNode(int, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >) (part_c.cpp:56)
==11206==    by 0x10944D: main (part_c.cpp:92)
==11206==
==11206== Invalid read of size 8
==11206==    at 0x49C8AE4: std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >::~basic_string() (in /usr/lib/x86_64-linux-gnu/libstdc++.so.6.0.33)
==11206==    by 0x1095D9: ListNode::~ListNode() (part_c.cpp:5)
==11206==    by 0x109611: NodeList::freeChain(ListNode*) (part_c.cpp:32)
==11206==    by 0x109681: NodeList::~NodeList() (part_c.cpp:44)
==11206==    by 0x1094A5: main (part_c.cpp:100)
==11206==  Address 0x4e1e168 is 8 bytes inside a block of size 48 free'd
==11206==    at 0x484A61D: operator delete(void*, unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==11206==    by 0x109494: main (part_c.cpp:97)
==11206==  Block was alloc'd at
==11206==    at 0x4846FA3: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==11206==    by 0x1096AB: NodeList::addNode(int, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >) (part_c.cpp:56)
==11206==    by 0x10944D: main (part_c.cpp:92)
==11206==
```

```
==11206==
==11206== Invalid free() / delete / delete[] / realloc()
==11206==    at 0x484A61D: operator delete(void*, unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==11206==    by 0x10961E: NodeList::freeChain(ListNode*) (part_c.cpp:32)
==11206==    by 0x109681: NodeList::~NodeList() (part_c.cpp:44)
==11206==    by 0x1094A5: main (part_c.cpp:100)
==11206==  Address 0x4e1e160 is 0 bytes inside a block of size 48 free'd
==11206==    at 0x484A61D: operator delete(void*, unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==11206==    by 0x109494: main (part_c.cpp:97)
==11206==  Block was alloc'd at
==11206==    at 0x4846FA3: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==11206==    by 0x1096AB: NodeList::addNode(int, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >) (part_c.cpp:56)
==11206==    by 0x10944D: main (part_c.cpp:92)
==11206==
==11206==
==11206== HEAP SUMMARY:
==11206==     in use at exit: 0 bytes in 0 blocks
==11206==   total heap usage: 4 allocs, 5 frees, 73,872 bytes allocated
==11206==
==11206== All heap blocks were freed -- no leaks are possible
==11206==
==11206== For lists of detected and suppressed errors, rerun with: -s
==11206== ERROR SUMMARY: 3 errors from 3 contexts (suppressed: 0 from 0)
amydayana@amy:~/eece2140/Week 11 Lab Assignment$
```

Bug 3

```
amydayana@amy:~/eece2140/Week 11 Lab Assignment$ valgrind --leak-check=full --track-origins=yes ./part_c
==12257== Invalid read of size 1
==12257==    at 0x4853958: mempcpy (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==12257==    by 0x4B9FA3F: _IO_new_file_xsputn (fileops.c:1236)
==12257==    by 0x4B9FA3F: _IO_file_xsputn@@GLIBC_2.2.5 (fileops.c:1197)
==12257==    by 0x4B92A11: fwrite (iofwrite.c:39)
==12257==    by 0x49B6DC3: std::basic_ostream<char, std::char_traits<char> >& std::__ostream_insert<char, std::char_traits<char> >(std::basic_ostream<char, std::char_traits<char> >&, char const*, long)
 (in /usr/lib/x86_64-linux-gnu/libstdc++.so.6.0.33)
==12257==    by 0x10A505: main (part_c.cpp:98)
==12257==  Address 0x4e1e10b is 27 bytes inside a block of size 48 free'd
==12257==    at 0x484A61D: operator delete(void*, unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==12257==    by 0x10A829: NodeList::removeNode(int) (part_c.cpp:71)
==12257==    by 0x10A495: main (part_c.cpp:94)
==12257==  Block was alloc'd at
==12257==    at 0x4846FA3: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==12257==    by 0x10A70F: NodeList::addNode(int, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >) (part_c.cpp:56)
==12257==    by 0x10A45B: main (part_c.cpp:91)
==12257==
==12257== Invalid read of size 1
==12257==    at 0x4853965: mempcpy (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==12257==    by 0x4B9FA3F: _IO_new_file_xsputn (fileops.c:1236)
==12257==    by 0x4B9FA3F: _IO_file_xsputn@@GLIBC_2.2.5 (fileops.c:1197)
==12257==    by 0x4B92A11: fwrite (iofwrite.c:39)
==12257==    by 0x49B6DC3: std::basic_ostream<char, std::char_traits<char> >& std::__ostream_insert<char, std::char_traits<char> >(std::basic_ostream<char, std::char_traits<char> >&, char const*, long)
 (in /usr/lib/x86_64-linux-gnu/libstdc++.so.6.0.33)
==12257==    by 0x10A505: main (part_c.cpp:98)
==12257==  Address 0x4e1e109 is 25 bytes inside a block of size 48 free'd
==12257==    at 0x484A61D: operator delete(void*, unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==12257==    by 0x10A829: NodeList::removeNode(int) (part_c.cpp:71)
==12257==    by 0x10A495: main (part_c.cpp:94)
==12257==  Block was alloc'd at
==12257==    at 0x4846FA3: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==12257==    by 0x10A70F: NodeList::addNode(int, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >) (part_c.cpp:56)
==12257==    by 0x10A45B: main (part_c.cpp:91)
==12257==
name: Dean
==12257==
==12257== HEAP SUMMARY:
==12257==     in use at exit: 0 bytes in 0 blocks
==12257==   total heap usage: 4 allocs, 4 frees, 74,848 bytes allocated
==12257==
==12257== All heap blocks were freed -- no leaks are possible
==12257==
==12257== For lists of detected and suppressed errors, rerun with: -s
==12257== ERROR SUMMARY: 11 errors from 6 contexts (suppressed: 0 from 0)
amydayana@amy:~/eece2140/Week 11 Lab Assignment$
```

Written Comparisons

Lines of Code:

Between parts a and b, a was much longer. It had around 30 more lines of code compared to part b because it had the destructor, copy constructor, assignment operator, and then functions that helped it run. However, in part b, unique_ptr took care of all of that, making the amount of code much less than that of part a. As a result, it was able to reduce the lines of code needed and therefore, let the programmer be more efficient.

Safety:

Part a deals with more errors in comparison to part b because it had to create the destructor, copy constructor, assignment operator, etc, whereas part b had that handled through unique_ptr. As a result, part b was able to manage leaks and double frees. Part c highlights the effects of bugs through common mistakes that could happen in code such as part a. These errors don't directly affect the output, but it creates chaos for the memory management.

Complexity:

Part a is much more complex because the coder has to carefully make sure each aspect of the code is written down, or else it would create memory issues. Some mistakes are easy to make and hard to catch, as seen in part c where some of those issues are highlighted. The worst part is that these errors still compile the code without any issues. On the other hand, part b handles those issues through unique_ptr, ensuring that certain mistakes, as seen in part c, are taken care of once it compiles rather than continuing to run with those issues.